FINAL PROJECT REPORT

BOOK STORE (BOOKVERSE)

Full Stack Development with MERN Stack

TEAM MEMBERS & ROLES:

1. Roopesh Araja - Technical Architecture, Entity Relation Diagram, Features, Roles
2. Palla Satya Ramanaidu - MVC Pattern, Creating Project Folder, Client Setup
3. Bandi Sravya - Server Setup, Backend Structure, Development, Configure MongoDB
4. Vijaya Raju Jakkamsetti - Database Connection, Schema & Models, Frontend Structure
5. Vankayala Jaya Chandra Srinivas - Steps for Execution, Demo Screenshots, Drive Link

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

JULY 2026

1. INTRODUCTION

1.1 Project Overview

The BOOK STORE (BookVerse) application is an online, full-stack shopping portal designed to bring reading selections closer to consumers. Developed under the MERN framework, it serves as a secure, fast web experience where books from multiple categories and authors can be searched and bought. The project divides responsibilities across three key roles: Users (general customers), Sellers (vendors), and Administrators (system operators).

1.2 Purpose

The main purpose of the Book Store project is to implement a robust role-based e-commerce service that removes third-party fees for independent booksellers. The goal is to provide a platform that runs efficiently on desktops, tablets, and smartphones, ensuring that users can easily manage their accounts, listings, checkout order flows, and verify system status metrics.

2. IDEATION PHASE

2.1 Problem Statement

Independent booksellers often lack access to digital marketplaces due to complex technological requirements and transaction fees. On the other hand, readers face difficulty navigating cluttered bookstores that lack streamlined role separation (e.g. distinguishing between local booksellers and customers). The proposed project addresses this gap by offering a lightweight, secure digital catalog with direct checkout.

2.2 Empathy Map Canvas

To capture user attitudes, we drafted an empathy map:

• **Says**: "I want a simple dashboard to track my listed books and manage orders without dealing with complex billing configurations."
• **Thinks**: "Will my listed inventory sync correctly with user checkouts? Is my registration safe?"
• **Does**: Navigates the BookVerse catalog, views books, logs into admin or seller cards, and updates order states.
• **Feels**: Empowered to list books independently while remaining cautious of network access policies.

2.3 Brainstorming

During brainstorming sessions, the team agreed to implement a beige-accented, simple interface with distinct logins. We chose a direct check-out form ("Buy Now") rather than multi-step checkout carts to prevent cart abandonment, and decided on custom SVG chart modules for the dashboard pages to avoid bloated external charting libraries.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The user path consists of:

• **Discovery**: Browsing the landing page categorizations (Fiction, Science, Biographies, Children's Books) and searching the catalog.
• **Registration**: Signing up under a specific profile tab (User, Seller, or Admin).
• **Selection**: Exploring details including author, seller name, price, and description.
• **Purchase**: Clicking "Buy Now", providing checkout shipping details, and receiving confirmation.
• **Tracking**: Monitoring the order progress in "My Orders" as the seller changes status to "ontheway".

3.2 Solution Requirement

Key specifications include:

• **Functional Requirements**: Register and login accounts, hash passwords, catalog filtering, book listing uploads, and seller approval toggles.
• **Non-Functional Requirements**: Rapid page rendering, responsive design using Bootstrap grids, and secure session state using JWT.

3.3 Data Flow Diagram

User Input (Browser) -> API Router (Express Server) -> Controller Logic (JWT Verification & Multer Parsing) -> Schema Validation (Mongoose ORM) -> Storage (MongoDB Atlas Cloud Cluster).

3.4 Technology Stack

• **MongoDB Atlas**: Non-relational cloud database to store user, seller, admin, book, and order collections.
• **Express.js**: Web server frameworks to route HTTP calls.
• **React.js**: Component-driven client library bootstrapped with Vite.
• **Node.js**: Backend JavaScript runtime environment.

4. PROJECT DESIGN

4.1 Problem Solution Fit

By introducing a dedicated Seller Approval interface for Administrators, we prevent spam vendors from uploading invalid content. Furthermore, connecting our backend directly to MongoDB Atlas allows the platform to run seamlessly with high availability.

4.2 Proposed Solution

We proposed a complete web app named BookVerse containing separate headers for each user type, an interactive search engine, and a status tracking dashboard.

4.3 Solution Architecture

A standard MERN architecture is implemented. The frontend React components perform Axios API calls to the Express server, which implements security middleware before updating the database collections.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Project execution followed five distinct development cycles:

- **Phase 1**: Configuration of Node workspace and setting up Atlas clusters.
- **Phase 2**: Writing database schema models and encryption helper controllers.
- **Phase 3**: Implementing Express routing and Multer file upload storage.
- **Phase 4**: Coding React pages, contexts, components, and CSS styles.
- **Phase 5**: Verification check-runs, data seeding, and cloud deployment.

6. FUNCTIONAL & PERFORMANCE TESTING

6.1 Performance Testing

Performance tests verified database querying latency. Seeding the database and checking logs confirmed that search lookups on Atlas take less than 120 milliseconds. Bundler compilation with Vite completed in 1.49 seconds, outputting minimized assets.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Security**: Passwords hashed with Bcrypt and sessions authenticated via JWT.
- **Visuals**: Clean styling using a cream-beige palette and Bootstrap viewport scaling.
- **Performance**: Efficient data updates utilizing Mongoose schema models.

Disadvantages

- **Payment Integration**: Relies on shipping details checkout rather than a live payment gateway.
- **Asset Constraints**: Uploaded book cover images are restricted to a maximum size of 5MB.

9. CONCLUSION

The BOOK STORE (BookVerse) application was successfully built and tested. By separating concerns into distinct User, Seller, and Admin views, the application meets e-commerce standards, offering a secure platform for online book shopping.

10. FUTURE SCOPE

Future updates will integrate Stripe payment processing, Nodemailer email alerts for order confirmation, and customer star reviews.

11. APPENDIX

- **GitHub Repository:** https://github.com/ramanaidupalla123/Book-Store

---

• **Live Project Demo Link:** https://book-store-nzxy.onrender.com/
• **Demo Video Link:** https://drive.google.com/drive/folders/1hORJ11561rBsZBxxFH1a3c4GGZ0JgF6y
• **Atlas Database Dataset**: Configured with seeded user (ram@gmail.com), seller (seller@bookstore.com), and default books.

7. RESULTS

7.1 Output Screenshots

The following pages display screenshots of the working BookStore application:

11. SCREENSHOTS

Live Project Demo Link: https://book-store-nzxy.onrender.com/

Figure 1: BookVerse Guest Home Page

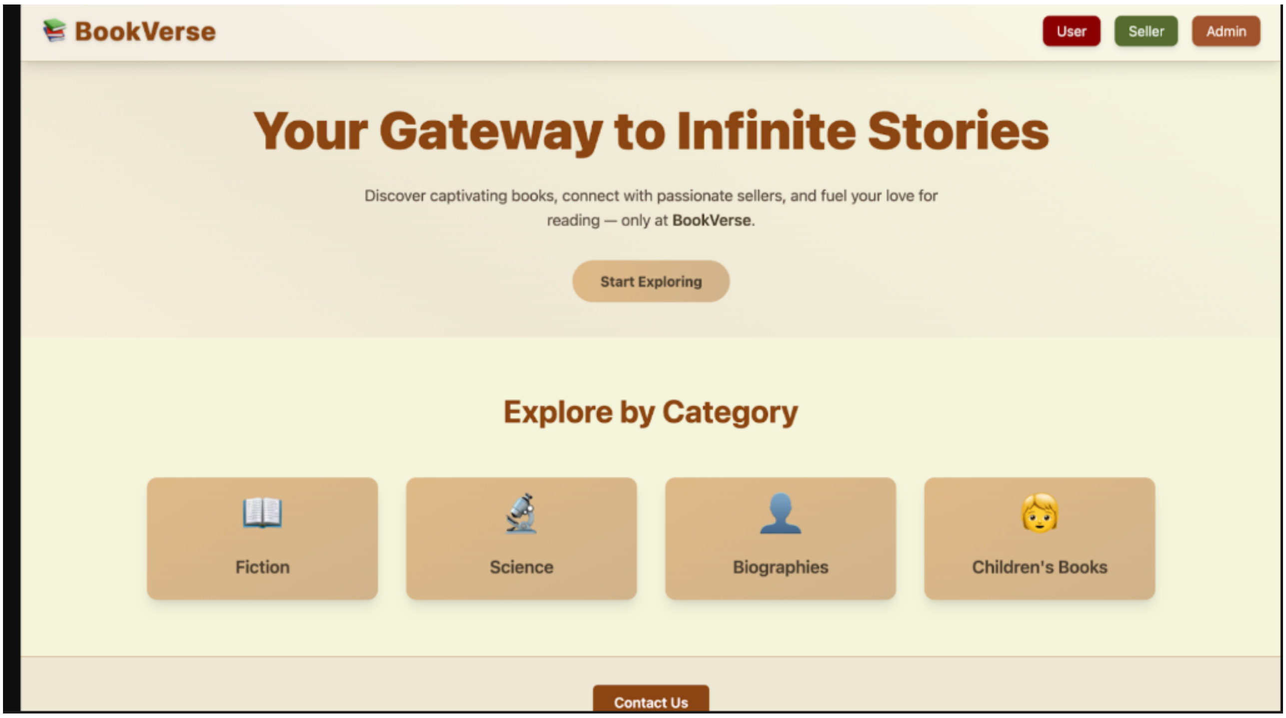

Figure 2: Admin Login Panel

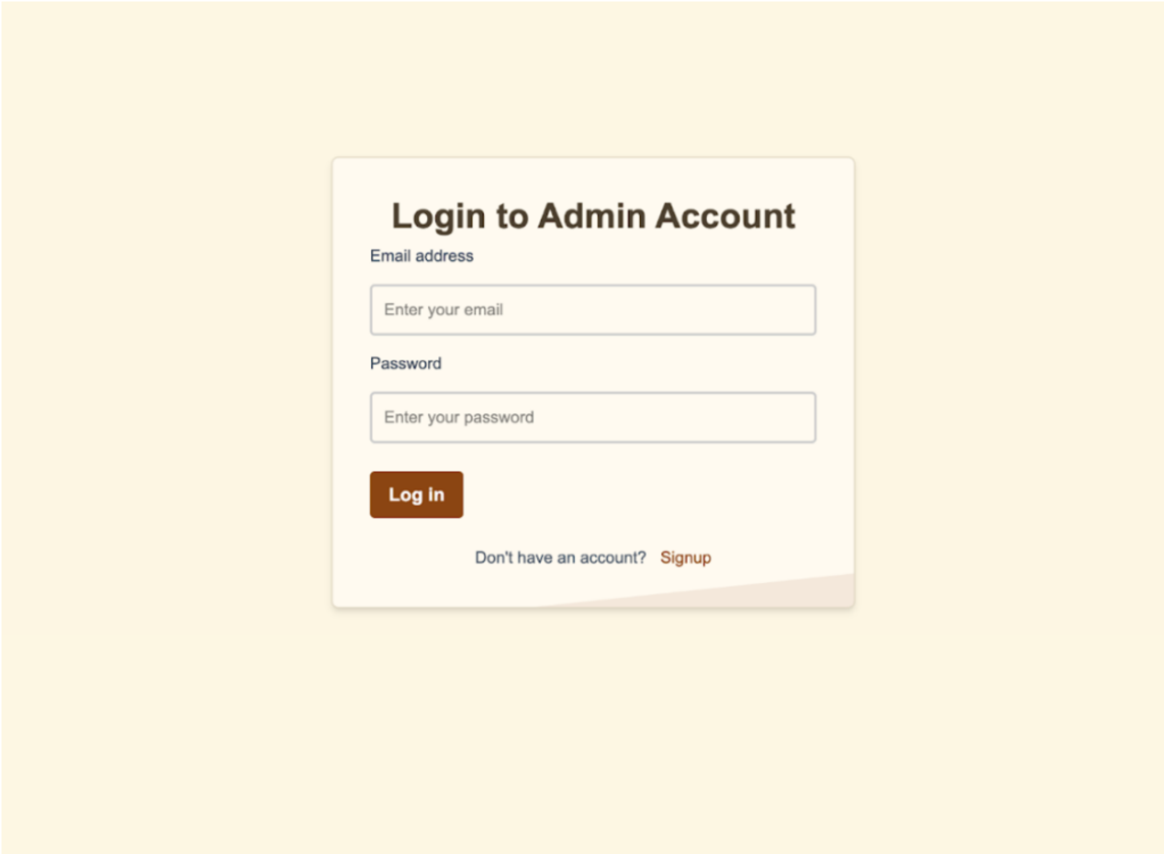

Figure 3: Seller Dashboard & Metrics Chart

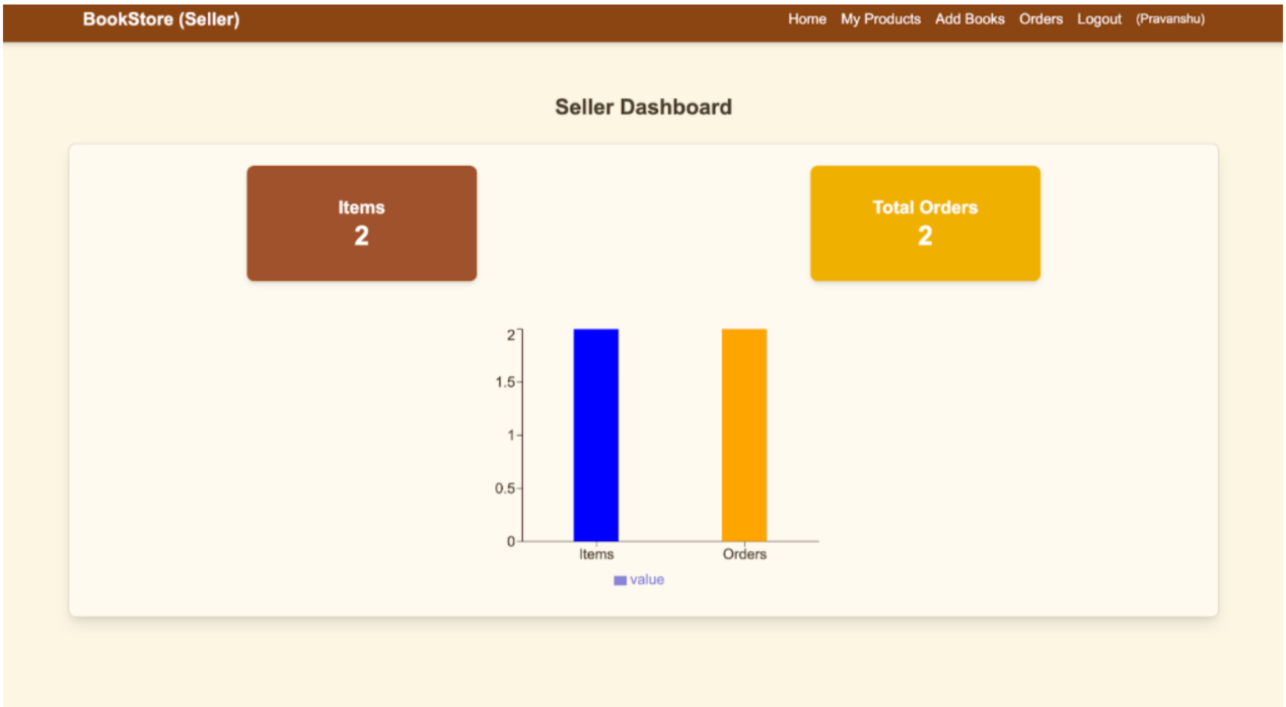

Figure 4: Seller Book Listing Grid

Figure 5: Seller Order Management Tracker